EECS 482

Lab 2: Monitors, Automated Testing

Administrivia

Today: 05/23

Project 1 Due &
Project 2 Release
06/2

Those not in groups will work solo

Agenda

1. Condition Variables
2. Q1: Ping Pong Exercise
3. Q2: Piano Exercise
4. Q3: Correctness Checker

Condition Variables

Recall: **Synchronization** prevents race conditions.

We need to manage how multiple threads are allowed to access certain things (shared data).

1. **Mutual exclusion**: only one thread can do this at a time (mutex)
2. **Ordering**: this thread must wait until some condition is true (condition variable)

Condition variables provide ordering for threads

- Think of it like a waiting room.
- **cv::wait(mutex &m)**
  - Adds current thread to the waiting room.
- **cv::signal()**
  - Removes one thread from waiting room.
- **cv::broadcast()**
  - Removes all threads from waiting room.

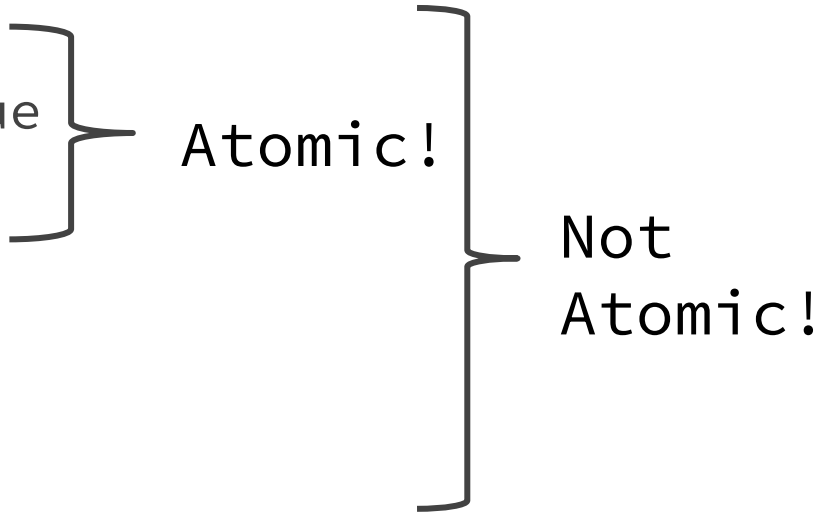
CVs: what does wait(m) do?

---

cv.wait(m) does four things:

1. Release lock
2. Put thread on waiting queue
3. Go to sleep
   ...

4. Reacquire lock



Monitor Programming "Template"

```
 ___
{
  lock

  while (!stop_waiting_1) { wait }
  do stuff_1
  signal/broadcast anyone who cares about what we did

  while (!stop_waiting_2) { wait }
  do stuff_2
  signal/broadcast anyone who cares about what we did

  ...

  unlock
}
```

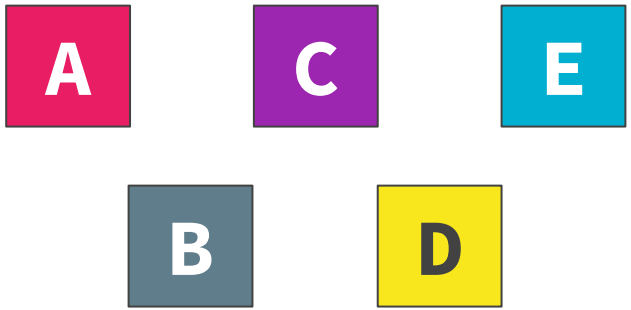
Programming with monitors

---

1. Identify **shared data**.
2. Assign **locks** to shared data.
3. Identify the **waiting conditions** of the threads.
4. Assign **condition variables** for each situation.
5. Use `while (!stop_waiting) { wait }`
6. Call `signal()/broadcast()` when threads modify data that may allow another thread to continue.

Locks

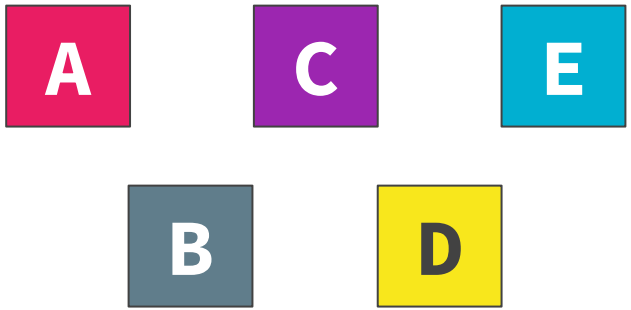
Does more concurrency (lock) imply better performance?



Locks

Does more concurrency (e.g., finer-grained locks) imply better performance?

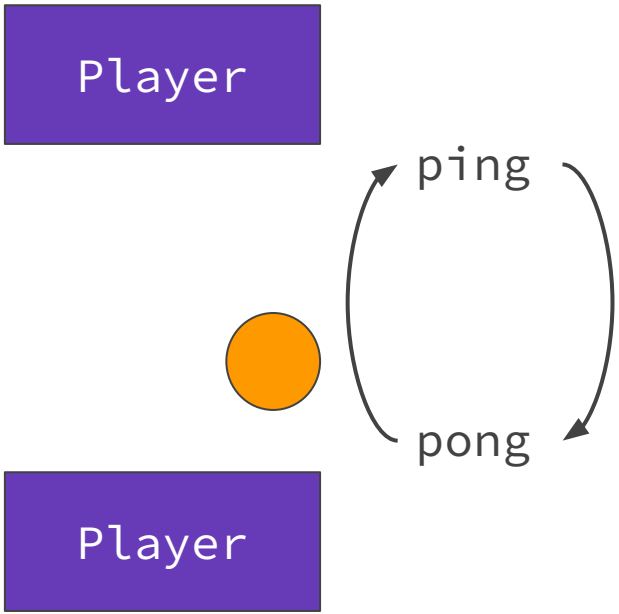
Not necessarily, there's overhead to locking and unlocking.



Q1: Ping Pong

Ping Pong Specification

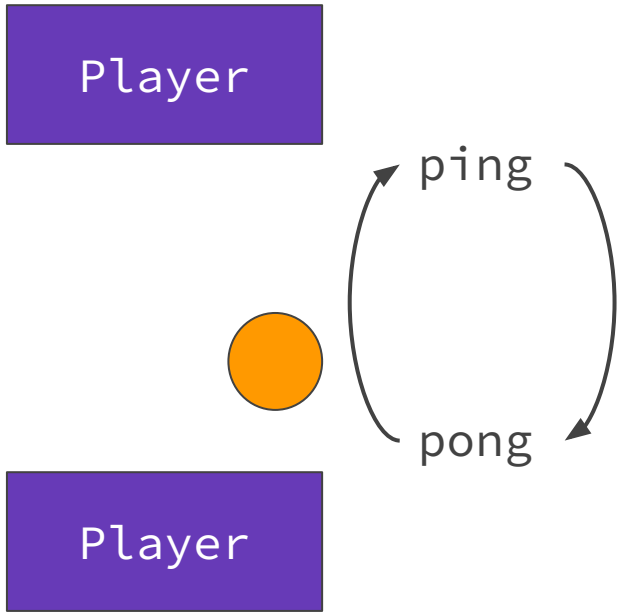
- One thread for each player
- Players must take turns hitting the ball
- Either player can start
- Each player hits ball 5 times
- Command-line argument to control determinism



Correct Behavior

---

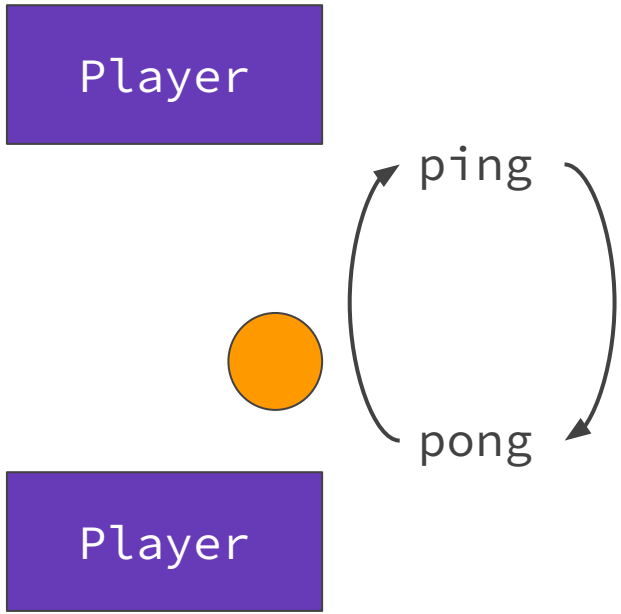
ping
pong
ping
pong
ping
pong
ping
pong
ping
pong



Correct Behavior

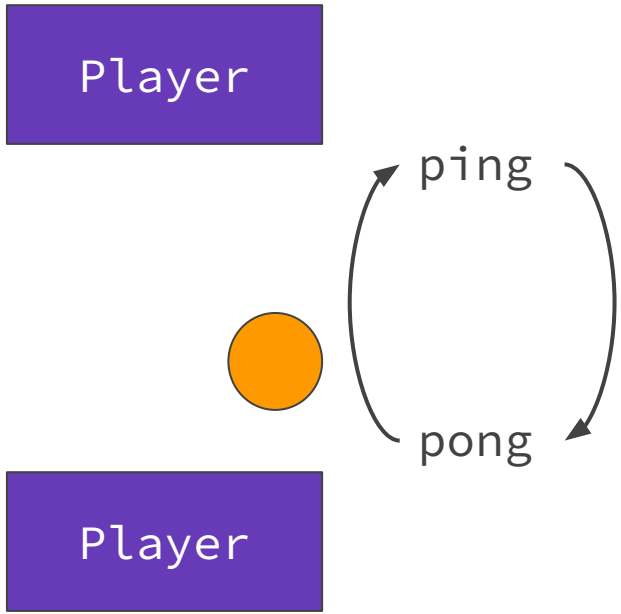
---

pong
ping
pong
ping
pong
ping
pong
ping
pong
ping



Incorrect Behavior

```
pong
ping
pong
pong
ping
ping
pong
ping
pong
ping
```



Lab Exercise: Implement Ping Pong

- Use Project 1 Infrastructure
- `g++ -std=c++20 pingpong.cpp -ldl -pthread libthread.o -o pingpong`
  - `./pingpong`
- **2 threads, taking turns, 5 times**
- **Command-line argument for determinism**

Programming with monitors

---

1. Identify **shared data**.
2. Assign **locks** to shared data.
3. Identify the **waiting conditions** of the threads.
4. Assign **condition variables** for each situation.
5. Use `while (!stop_waiting) { wait }`
6. Call `signal()/broadcast()` when threads modify data that may allow another thread to continue.

Shared data, locks, waiting conditions

---

1. Identify **shared data**.
    - **What's shared data here?**

2. Assign **locks** to shared data.
    - **How many lock(s) do we need?**

3. Identify the **waiting conditions** of the threads.
    - **What are the waiting conditions?**
    - **Try to list them for each thread.**

Shared data, locks, waiting conditions (answers)

---

1. Identify **shared data**.
   shared data: turn

2. Assign **locks** to shared data.
   turnMutex

3. Identify the **waiting conditions** of the threads.
   - **Player 0 waits for Player 1 to hit the ball**
   - **Player 1 waits for Player 0 to hit the ball**

Ping Pong Solution – Separate Local Variables

```
mutex turn_mutex;
int turn = 0;
cv c;
```

```
void pinger(uintptr_t) {
    int hit_count = 0;
    while (hit_count < 5) {
        turn_mutex.lock();
        std::cout<<"ping"<<std::endl;
        turn = 1;
        c.signal();
        while (turn != 0) {
            c.wait(turn_mutex);
        }
        hit_count++;
        turn_mutex.unlock();
    }
}
```

```
void ponger(uintptr_t) {
    int hit_count = 0;
    while (hit_count < 5) {
        turn_mutex.lock();
        std::cout<<"pong"<<std::endl;
        turn = 0;
        c.signal();
        while (turn != 1) {
            c.wait(turn_mutex);
        }
        hit_count++;
        turn_mutex.unlock();
    }
}
```

Ping Pong Solution – one Turn Variable, condensed

```
void player(uintptr_t arg) {
    while (true) {
        turn_mutex.lock();
        std::cout<< arg % 2 ? "pong" : "ping" <<std::endl;
        turn++;
        c.signal();
        while (turn % 2 != arg) {
            c.wait(turn_mutex);
        }
        if (turn > 9) {
            // break with deadlock
            break;
        }
        turn_mutex.unlock();
    }
}
```

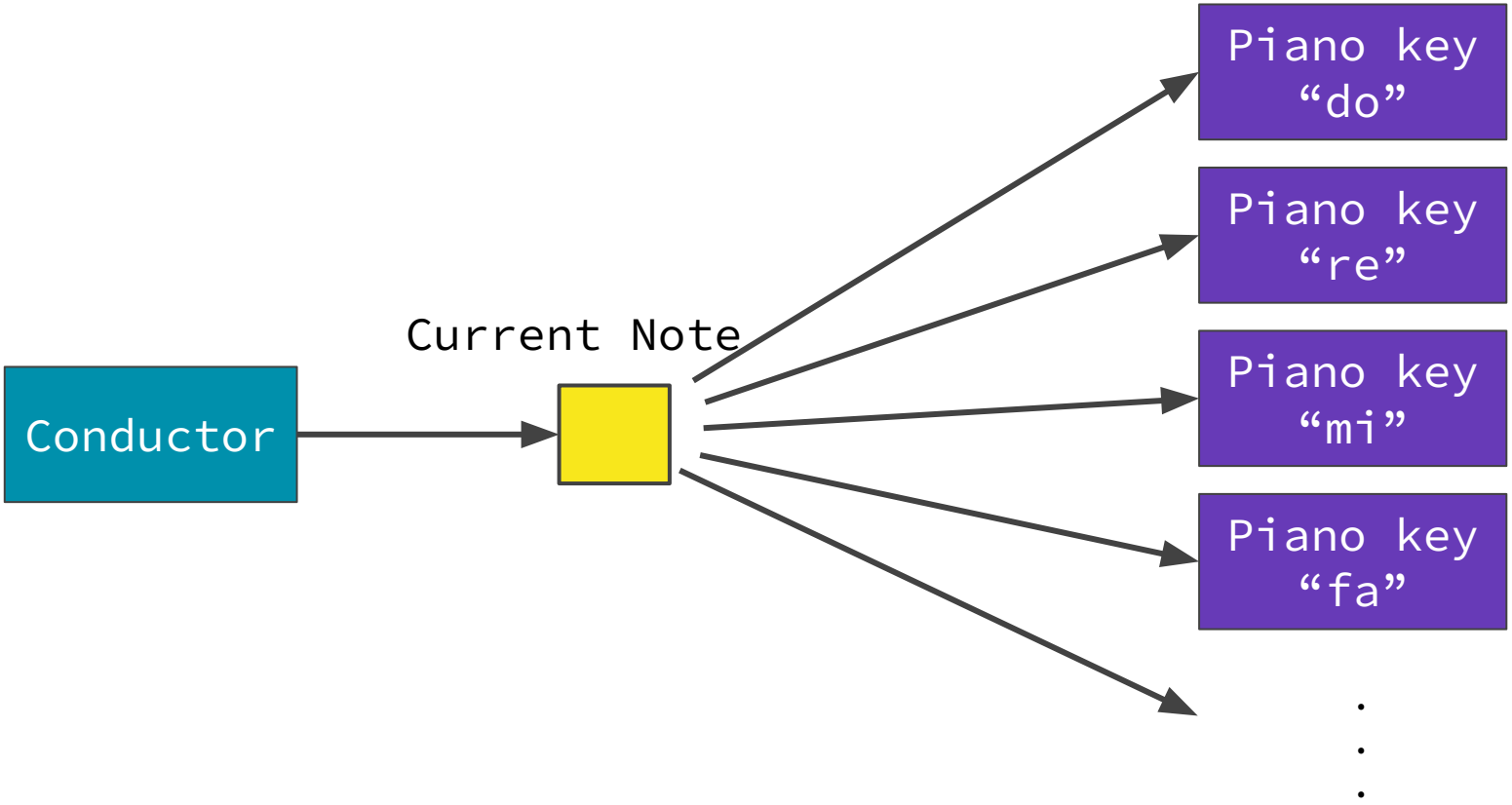
```
mutex turn_mutex;
int turn = 0;
cv c;
```

```
void start(uintptr_t) {
    thread pinger(player, 0);
    thread ponger(player, 1);
}

int main(int argc, char* argv[]) {
    assert(argc == 2);
    cpu::boot(start, 0, atoi(argv[1]));
}
```

Q2: Piano Example

Programming with monitors: Piano example



Lab exercise: implement conductor() and pianoKey()

```
// “na” represent empty note
enum Note { na = 0, d0 = 1, re = 2, mi = 3, fa = 4, ...};
void play(Note i); // helper function

void conductor();
void pianoKey(Note i);
```

Handout in Lab Exercises!

Programming with monitors

---

1. Identify **shared data**.
2. Assign **locks** to shared data.
3. Identify the **waiting conditions** of the threads.
4. Assign **condition variables** for each situation.
5. Use `while (!stop_waiting) { wait }`
6. Call `signal()/broadcast()` when threads modify data that may allow another thread to continue.

Shared data, locks, waiting conditions

---

1. Identify **shared data**.
    - **What's shared data here?**

2. Assign **locks** to shared data.
    - **How many lock(s) do we need?**

3. Identify the **waiting conditions** of the threads.
    - **What are the waiting conditions?**
    - **Try to list them for each thread.**

Shared data, locks, waiting conditions (answers)

---

1. Identify **shared data**.
   shared data: current note

2. Assign **locks** to shared data.
   noteMutex

3. Identify the **waiting conditions** of the threads.
   - **Conductor cannot conduct if current note is not empty**
   - **Piano key cannot play if current note is not THIS key**

How many cv(s) to use?

---

4. Assign **condition variables** for each situation.
    - **How many cv(s) do we need?**

5. Use `while (!stop_waiting) { wait }`

6. Call `signal()/broadcast()` when threads modify data that affects the conditions.
    - **Conductor changes current note from empty to actual note**
    - **Piano keys change the current note to empty note**

First attempt - 1 condition variable

```
void conductor(){
    Note tmpNote;
    std::ifstream infile("input.txt");
    while(infile >> tmpNote){
        noteMutex.lock();

        while(currentNote != na){
            pianoCv.wait(noteMutex);
        }

        // change shared state
        // broadcast or signal

        noteMutex.unlock();
    }
}
```

```
void pianoKey(Note myNote){
    while (true) {
        noteMutex.lock();

        while (currentNote != myNote) {
            pianoCv.wait(noteMutex);
        }

        play(myNote);

        // change shared state
        // broadcast or signal

        noteMutex.unlock();
    }
}
```

First attempt - 1 condition variable

```
void conductor(){
    Note tmpNote;
    std::ifstream infile("input.txt");
    while(infile >> tmpNote){
        noteMutex.lock();

        while(currentNote != na){
            pianoCv.wait(noteMutex);
        }

        currentNote = tmpNote;
        pianoCv.broadcast();

        noteMutex.unlock();
    }
}
```

```
void pianoKey(Note myNote){
    while (true) {
        noteMutex.lock();

        while (currentNote != myNote) {
            pianoCv.wait(noteMutex);
        }

        play(myNote);

        currentNote = Note::na;
        pianoCv.broadcast();

        noteMutex.unlock();
    }
}
```

Main disadvantages of this implementation

- **Poor readability**: using one cv for two different waiting conditions. Also making it hard to reason about correctness
- **Unnecessary wakeups**: when a thread wakes up from waiting on a condition variable that's been signaled, only to discover that the condition it was waiting for isn't satisfied.

Disadvantages of first attempt

```
void conductor(){
    Note tmpNote;
    std::ifstream infile("input.txt");
    while(infile >> tmpNote){
        noteMutex.lock();

        while(currentNote != na){
            pianoCv.wait(noteMutex);
        }


        currentNote = tmpNote;
        pianoCv.broadcast();

        noteMutex.unlock();
    }
}
```

```
void pianoKey(Note myNote){
    while (true) {
        noteMutex.lock();

        while (currentNote != myNote) {
            pianoCv.wait(noteMutex);
        }

        play(myNote);

        currentNote = Note::na;
        pianoCv.broadcast();

        noteMutex.unlock();
    }
}
```

Disadvantages of first attempt

```
void conductor(){
    Note tmpNote;
    std::ifstream infile("input.txt");
    while(infile >> tmpNote){
        noteMutex.lock();

        while(currentNote != na){
            pianoCv.wait(noteMutex);
        }

        currentNote = tmpNote;
        pianoCv.broadcast();

        noteMutex.unlock();
    }
}
```

One waiting condition with this CV

```
void pianoKey(Note myNote){
    while (true) {
        noteMutex.lock();

        while (currentNote != myNote) {
            pianoCv.wait(noteMutex);
        }

        play(myNote);

        currentNote = Note::na;
        pianoCv.broadcast();

        noteMutex.unlock();
    }
}
```

Different waiting condition with same CV

Disadvantages of first attempt

```
void conductor(){
    Note tmpNote;
    std::ifstream infile("input.txt");
    while(infile >> tmpNote){
        noteMutex.lock();

        while(currentNote != na){
            pianoCv.wait(noteMutex);
        }

        currentNote = tmpNote;
        pianoCv.broadcast();

        noteMutex.unlock();
    }
}
```

One waiting condition with this CV

```
void pianoKey(Note myNote){
    while (true) {
        noteMutex.lock();

        while (currentNote != myNote) {
            pianoCv.wait(noteMutex);
        }

        play(myNote);

        currentNote = Note::na;
        pianoCv.broadcast();

        noteMutex.unlock();
    }
}
```

Different waiting condition with same CV

Wakes up all waiting piano keys and conductor, but only the conductor can make progress

How do we fix this?

- **Use multiple CVs!**
- conductorCv
  - Conductor thread waits on this cv
  - Piano keys will wake conductor up by signalling or broadcasting
- pianoCv
  - Piano key threads wait on this cv
  - Conductor will wake piano keys up by signalling or broadcasting

Second attempt - 2 condition variables

```
void conductor(){
   Note tmpNote;
   std::ifstream infile("input.txt");
   while(infile >> tmpNote){
      noteMutex.lock();

      while(currentNote != na){
         conductorCv.wait(noteMutex);
      }

      currentNote = tmpNote;
      // signal or broadcast

      noteMutex.unlock();
   }
}
```

```
void pianoKey(Note myNote){
   while (true) {
      noteMutex.lock();

      while(currentNote != myNote) {
         pianoCv.wait(noteMutex);
      }

      play(myNote);

      currentNote = na;
      // signal or broadcast

      noteMutex.unlock();
   }
}
```

Second attempt - 2 condition variables

```
void conductor(){
   Note tmpNote;
   std::ifstream infile("input.txt");
   while(infile >> tmpNote){
      noteMutex.lock();

      while(currentNote != na){
         conductorCv.wait(noteMutex);
      }

      currentNote = tmpNote;
      pianoCv.broadcast();

      noteMutex.unlock();
   }
}
```

```
void pianoKey(Note myNote){
   while (true) {
      noteMutex.lock();

      while(currentNote != myNote) {
         pianoCv.wait(noteMutex);
      }

      play(myNote);

      currentNote = na;
      conductorCv.signal();

      noteMutex.unlock();
   }
}
```

Improvements from this implementation

- **Much better readability**: conductorCv for conductor's wait condition and pianoCv for piano keys' wait condition
- **Reduced unnecessary wakeups**: pianoKey threads only signal the conductor, they do not wake up the waiting piano keys
- **But, is there still inefficiency? Can we do better?**

Unnecessary Wakeups

```
void conductor(){
   Note tmpNote;
   std::ifstream infile("input.txt");
   while(infile >> tmpNote){
      noteMutex.lock();

      while(currentNote != na){
         conductorCv.wait(noteMutex);
      }

      currentNote = tmpNote;
      pianoCv.broadcast();

      noteMutex.unlock();
   }
}
```

```
void pianoKey(Note myNote){
   while (true) {
      noteMutex.lock();

      while(currentNote != myNote){
          pianoCv.wait(noteMutex);
      }

      play(myNote);

      currentNote = na;
      conductorCv.signal();

      noteMutex.unlock();
  }
}
```

Unnecessary Wakeups

```
void conductor(){
   Note tmpNote;
   std::ifstream infile("input.txt");
   while(infile >> tmpNote){
      noteMutex.lock();

      while(currentNote != na){
         conductorCv.wait(noteMutex);
      }

      currentNote = tmpNote;
      pianoCv.broadcast();

      noteMutex.unlock();
   }
}
```

wakes up all waiting piano keys, but only the piano key == currentNote can make progress

```
void pianoKey(Note myNote){
   while (true) {
      noteMutex.lock();

      while(currentNote != myNote){
          pianoCv.wait(noteMutex);
      }

      play(myNote);

      currentNote = na;
      conductorCv.signal();

      noteMutex.unlock();
  }
}
```

How do we fix this?

- Need finer granularity for piano keys
  - Need to be able to wake up **specific** piano keys
- **Use more CVs!**
- Give each piano key its own CV

Third Attempt - 1 CV per Piano Key

```
map<Note, cv> pianoCvs;
pianoCvs[d0] = cv();
pianoCvs[re] = cv();
pianoCvs[mi] = cv();
...
```

```
void conductor(){
   Note tmpNote;
   std::ifstream infile("input.txt");
   while(infile >> tmpNote){
       noteMutex.lock();

       while(currentNote != na){
         conductorCv.wait(noteMutex);
       }

       currentNote = tmpNote;
       pianoCvs[currentNote].signal();

       noteMutex.unlock();
   }
}
```

```
void pianoKey(Note myNote){
   while (true) {
       noteMutex.lock();

       while(currentNote != myNote){
          pianoCvs[myNote].wait(noteMutex);
       }

       play(myNote);

       currentNote = na;
       conductorCv.signal();

       noteMutex.unlock();
   }
}
```

Which implementation is best?

- First attempt - Not great
  - Poor readability
  - Poor efficiency
- Second attempt - Pretty good
  - Good readability
  - Some inefficiency
- Third attempt - Pretty good
  - Best efficiency
  - Harder to program correctly and understand
- Tradeoff exists between "optimal" performance and "optimal" ease of getting it correct.
  - There's often no single answer to "how many CVs should I use?"
  - There are multiple ways to design a program with monitors - evaluating their tradeoffs is key

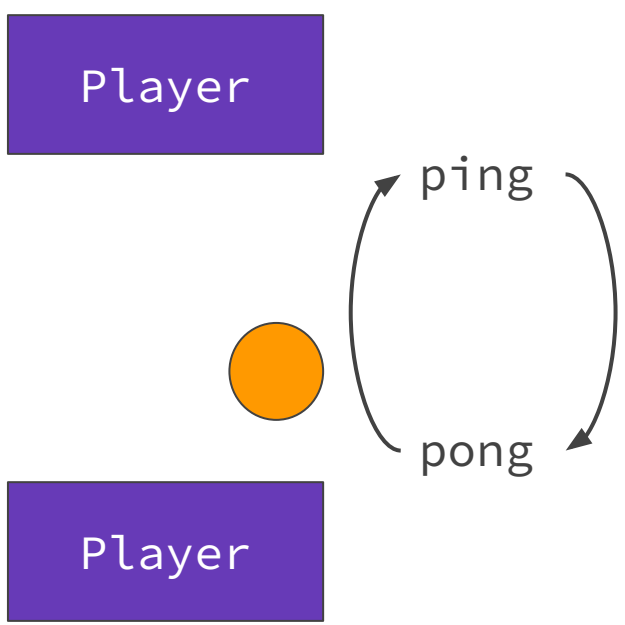
Correctness Checker

Why write a correctness checker?

- Automated testing
  - More complete and careful than manual checks
  - Allows for many more cases to be tested
    - Non determinism
    - Can combine with generated test cases
- Independent of implementation
  - Can test across multiple changes
- Used in industry (Regression test suites)

Q3: Correctness Checker for Ping Pong

- Write your own checker!
- Use a scripting language
- Recall ping pong spec
  - Either ping or pong can start
  - A ping-pong/pong-ping pair must occur before the next ping/pong. No double pings! (Or pongs!)
  - 5 turns per player
- Checker input is the output of your ping-pong program
- (optional) Step 0: Before writing a “complete” checker, what’s one easy thing you can check first?



pong
ping
pong
ping
pong
ping
pong
ping
pong
ping

```
def checker(filename):
    state = "start"
    ping_count = 0
    pong_count = 0
    with open(filename, 'r') as file:
        for line in file:
            if line == "ping\n":
                assert (state == "start" or state == "pong")
                state = "ping"
                ping_count += 1
            elif line == "pong\n":
                assert (state == "start" or state == "ping")
                state = "pong"
                pong_count += 1
            elif line == "No runnable threads.  Exiting.\n":
                # ignore infra line
                pass
            else:
                print("Invalid line:" + line)
                return
    assert (ping_count == 5)
    assert (pong_count == 5)
    print("All correct!")
```

```
if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Usage: python
script.py <filename>")
        sys.exit(1)

    filename = sys.argv[1]
    checker(filename)
```

For more complex outputs, regex is useful

Evaluating Correctness

Are either of these outputs incorrect? Why or why not?

```
driver 0 ready at (0,0)
driver 1 ready at (0,0)
customer 0 requests pizza at (0,4)
customer 1 requests pizza at (7,3)
customer 0 matched with driver 0
customer 1 matched with driver 1
driver 0 driving from (0,0) to (0,4)
driver 1 driving from (0,0) to (7,3)
customer 0 pays driver 0
customer 1 pays driver 1
customer 0 requests pizza at (5,0)
driver 0 ready at (0,4)
customer 0 matched with driver 0
driver 0 driving from (0,4) to (5,0)
driver 1 ready at (7,3)
customer 1 requests pizza at (1,0)
customer 0 pays driver 0
customer 1 matched with driver 1
driver 1 driving from (7,3) to (1,0)
driver 0 ready at (5,0)
customer 1 pays driver 1
driver 1 ready at (1,0)
No runnable threads.  Exiting.
```

```
driver 0 ready at (0,0)
driver 1 ready at (0,0)
customer 0 requests pizza at (0,4)
customer 0 matched with driver 0
driver 0 driving from (0,0) to (0,4)
customer 1 requests pizza at (7,3)
customer 0 pays driver 0
customer 1 matched with driver 1
driver 1 driving from (0,0) to (7,3)
driver 0 ready at (0,4)
customer 0 requests pizza at (5,0)
customer 1 pays driver 1
customer 0 matched with driver 0
driver 0 driving from (0,4) to (5,0)
driver 1 ready at (7,3)
customer 1 requests pizza at (1,0)
customer 0 pays driver 0
customer 1 matched with driver 1
driver 0 ready at (5,0)
driver 1 driving from (7,3) to (1,0)
customer 1 pays driver 1
driver 1 ready at (1,0)
No runnable threads.  Exiting.
```

Correctness Criteria for Project 1?

- Driver must be ready before being matched
- Driver must be matched before driving
- Driver must have arrived before customer pays them
- Driver must be paid before being ready
- Driver must be ready after being paid
- Driver must match with closest customer
- Customer must request pizza before being matched
- Driver must drive to matched customer
- Customer must pay matched driver
- Only matched customers and drivers should interact

Even More Automation

- Automatically generate tests, run them, and verify them.

```python
import random
import os

# run pizza and collect output
customers = ""
options = ["customer.in0", "customer.in1", "customer.in2", "customer.in3", "customer.in4"]

# choose a random number of customer input files
for _ in range(random.randint(1, 10)):
    customers += options[random.randint(1, 4)] + " "

# choose a random number of drivers
num_drivers = random.randint(1, 100)

# run pizza
os.system(f"./pizza {num_drivers} {customers} > output.txt")
print(f"Running ./pizza {num_drivers} {customers} > output.txt")

# read output
f = open("output.txt", "r")
lines = f.readlines()
f.close()

# run verifier on output...
```

More P1 Tips!

1. All of section 3.5 :)
2. “You may not assume anything about the order in which contending threads acquire the mutex.”
3. “threads may awaken spuriously” (cv wait)
4. “Threads may run at arbitrary speeds; you may not assume anything about their relative speeds.”
5. “Any thread may announce that a customer and driver have been matched by calling match.”
6. “A customer thread ends when all its requests have been completed. A driver thread never ends -- a driver's job is never done, and drivers never run out of pizza!”
7. “Driving is a slow operation, so any number of drivers must be able to execute drive at the same time.”
8. “plus 3 bonus feedbacks over the duration of this project.”

The spec has MANY more hints in it, continue to read it throughout the development process!

P1 Spec Hints (from last week)

- “Drivers and customers act by calling the following library functions, which are declared in pizza.h and defined in libthread.o. These functions are thread safe.”
- “Identify shared state, and assign a mutex to (each group of) this state. It is easier to write correct concurrent programs with fewer mutexes (though possibly with slower performance).?
- “wait should always be called within a loop that checks the condition being waited for.”
- “In addition to the autograder's evaluation of your program's correctness, a human grader will evaluate your program on issues such as compiler warnings, documentation, hard-coded constants, time and space efficiency, code duplication, and understandability. For documentation, you should at least write a block comment at the top of each function that explains the purpose and interface of that function.”

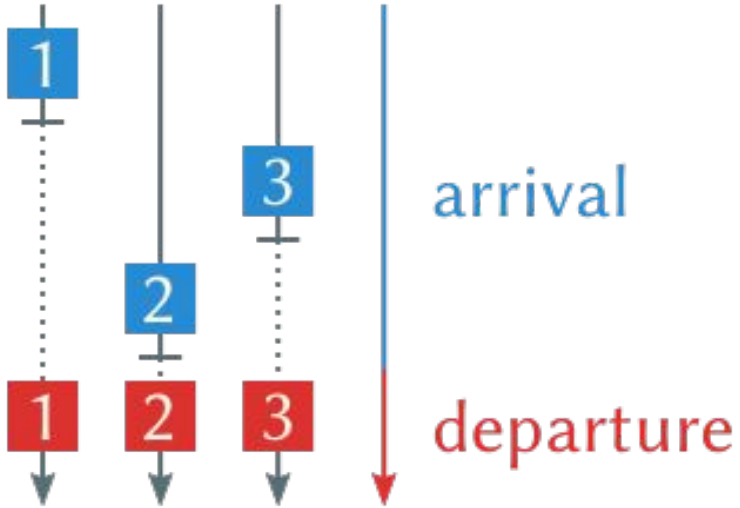
The spec has MANY more hints in it, continue to read it throughout the development process!

Q4: Monitor Barriers

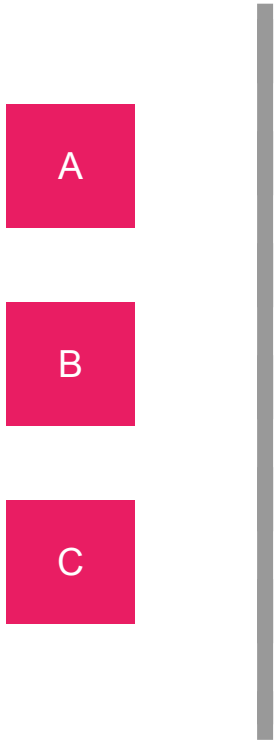
Barriers

Barriers allow a set of threads to synchronize their progress.

Threads "check in" or wait at a barrier and are only allowed to proceed past the barrier after all the threads have checked in.



Barriers



```
class Barrier {
 private:
  //Add your private member variables

 public:
  /* Constructs a new barrier that will allow
     number_of_threads threads to check in. */
  Barrier(int number_of_threads);

  /* Called by a thread checking in to the barrier. Should
     return true if the thread was the last thread to check
     in (in POSIX threads lingo, the "serial thread") and
     false otherwise. This function should block until all
     threads have checked in. */
  bool wait();
};
```

Example use of barrier: starting a race

```
int NUM_RUNNERS = 10;
Barrier starting_line(NUM_RUNNERS);

void runner_arrives() {
    // wait for all runners to arrive
    if (starting_line.wait()) {
        cout << "BANG!" << endl;
    }
    // run the race...
}
```

```
class Barrier {
 private:
  //Add your private member variables

 public:
  /* Constructs a new barrier that will allow
     number_of_threads threads to check in. */
  Barrier(int number_of_threads);

  /* Called by a thread checking in to the barrier. Should
     return true if the thread was the last thread to check
     in (in POSIX threads lingo, the "serial thread") and
     false otherwise. This function should block until all
     threads have checked in. */
  bool wait();
};
```

```
class Barrier {
 private:

                                          What's our shared data?



 public:
  /* Constructs a new barrier that will allow
     number_of_threads threads to check in. */
  Barrier(int number_of_threads) {


  }
```

```
class Barrier {
 private:
  mutex lock;
  cv cond;
  int num_threads;
  int checked_in;

 public:
  /* Constructs a new barrier that will allow
     number_of_threads threads to check in. */
  Barrier(int number_of_threads) {


  }
```

What's our shared data?

```
class Barrier {
 private:
  mutex lock;
  cv cond;
  int num_threads;
  int checked_in;

 public:
  /* Constructs a new barrier that will allow
     number_of_threads threads to check in. */
  Barrier(int number_of_threads) {
    num_threads = number_of_threads;
    checked_in = 0;
  }
```

```
// Returns true if this thread was the last to check-in
bool wait() {
  bool last = false;
  lock.lock();









  lock.unlock();
  return last;
}
```

```
// Returns true if this thread was the last to check-in
bool wait() {
  bool last = false;
  lock.lock();
  checked_in++;









  lock.unlock();
  return last;
}
```

```
// Returns true if this thread was the last to check-in
bool wait() {
  bool last = false;
  lock.lock();
  checked_in++;
  if (checked_in == num_threads) {
    cond.broadcast();
    last = true;
  }



  lock.unlock();
  return last;
}
```

We're the last thread, wake everyone up

```
// Returns true if this thread was the last to check-in
bool wait() {
  bool last = false;
  lock.lock();
  checked_in++;
  if (checked_in == num_threads) {
    cond.broadcast();
    last = true;
  } else {



  }
  lock.unlock();
  return last;
}
```

We're the last thread, wake everyone up

```
// Returns true if this thread was the last to check-in
bool wait() {
  bool last = false;
  lock.lock();
  checked_in++;
  if (checked_in == num_threads) {
    cond.broadcast();
    last = true;
  } else {
     while (checked_in < num_threads) {
       cond.wait(lock);
     }
  }
  lock.unlock();
  return last;
}
```

We're the last thread, wake everyone up

Sleep until everyone is here

```
// Returns true if this thread was the last to check-in
bool wait() {
  bool last = false;
  lock.lock();
  checked_in++;
  if (checked_in == num_threads) {
    cond.broadcast();
    last = true;
  } else {
     while (checked_in < num_threads) {
       cond.wait(lock);
     }
  }
  lock.unlock();
  return last;
}
```

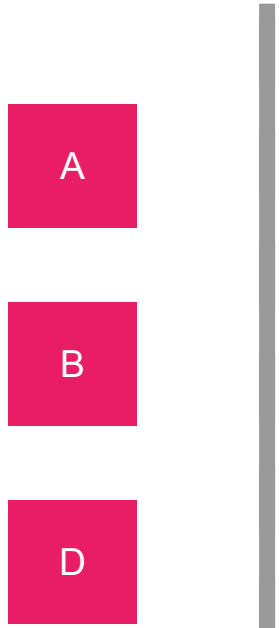
We're the last thread, wake everyone up

Sleep until everyone is here

How can we reuse this barrier?

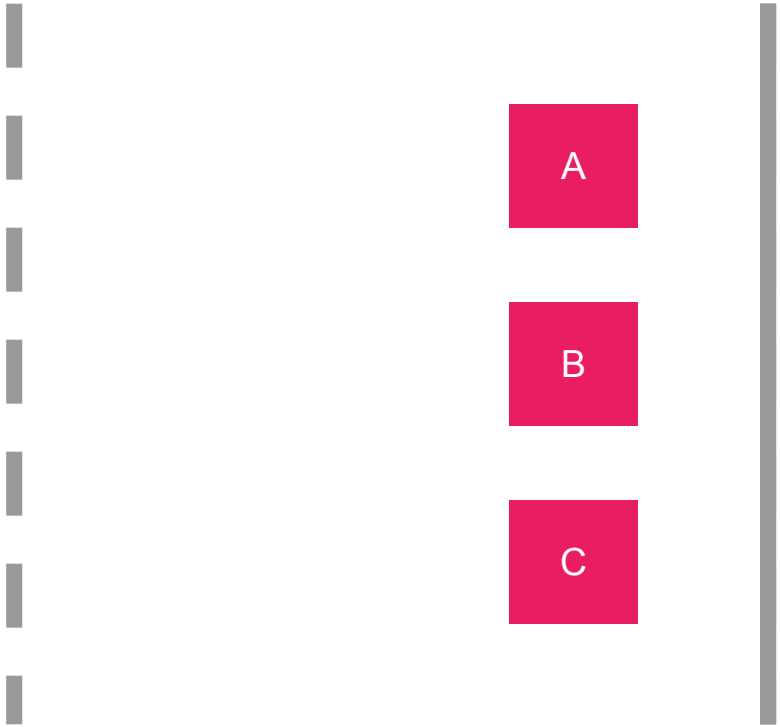
```
// Returns true if this thread was the last to check-in
bool wait() {
  bool last = false;
  lock.lock();
  checked_in++;
  if (checked_in == num_threads) {
    cond.broadcast();
    last = true;
    checked_in = 0;
  } else {
     while (checked_in < num_threads) {
       cond.wait(lock);
     }
  }
  lock.unlock();
  return last;
}
```

Does this work?


A
B
D

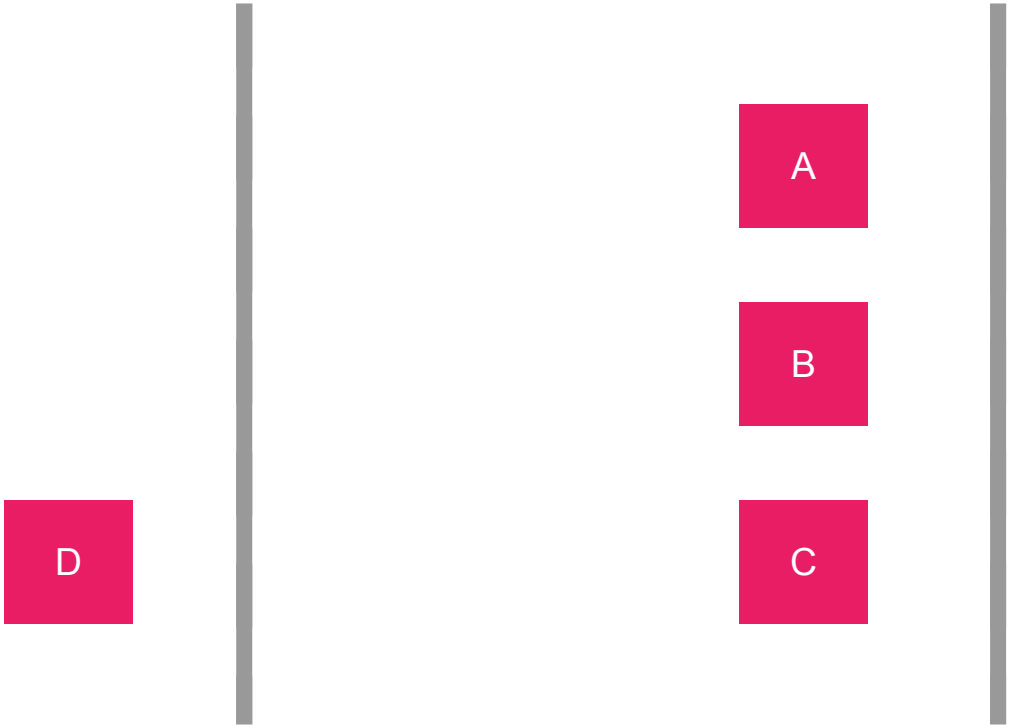
What is this solution lacking?

- A way to safely reset the barrier
  - Last thread to enter must allow others to exit (signal others)
  - Last thread to exit must prevent others from exiting (clean up)
- A way to prevent threads from entering while others are exiting
  - Last thread to enter must prevent others from entering
  - Last thread to exit must allow others to enter
  - 2 barriers?


A
B
C

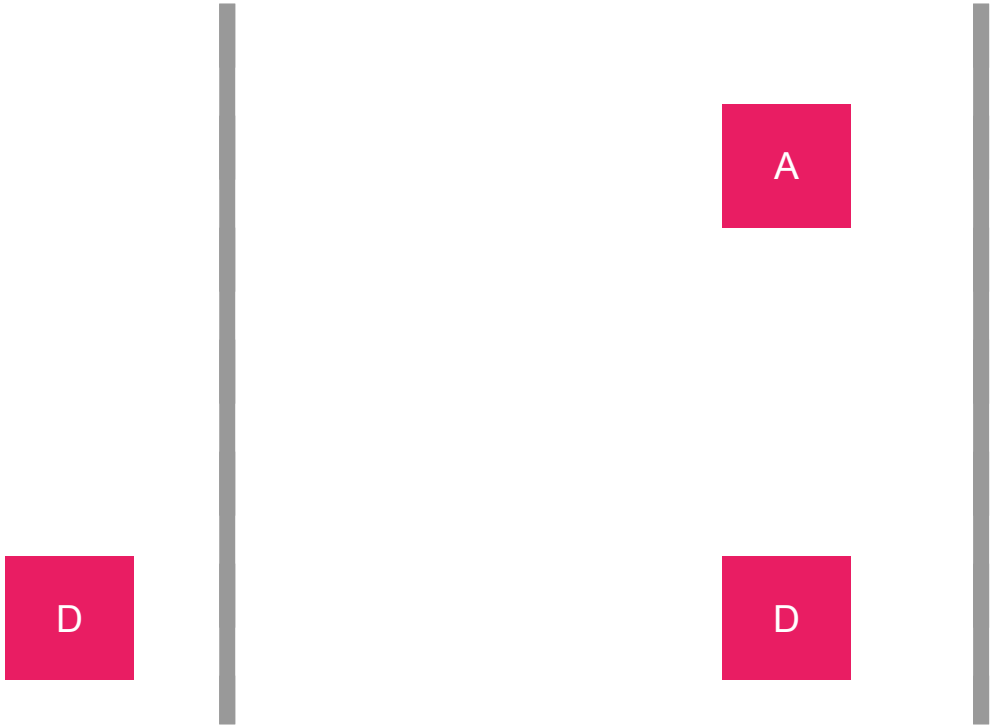
What is this solution lacking?

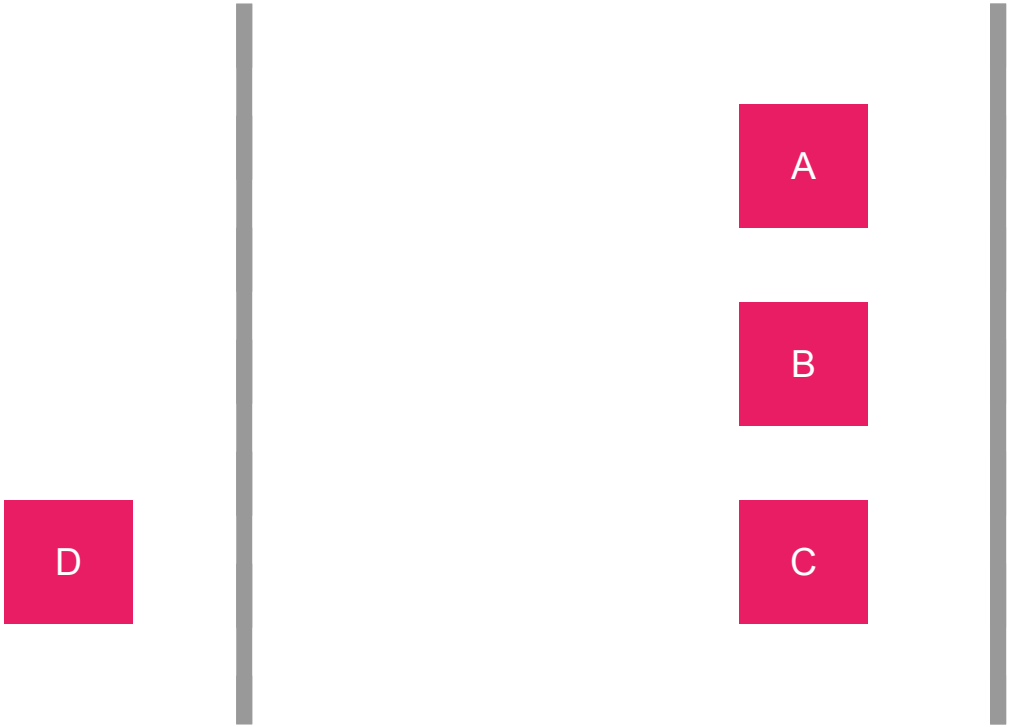
- A way to safely reset the barrier
  - Last thread to enter must allow others to exit (signal others)
  - Last thread to exit must prevent others from exiting (clean up)
- A way to prevent threads from entering while others are exiting
  - Last thread to enter must prevent others from entering
  - Last thread to exit must allow others to enter
  - 2 barriers?


A
B
C
D

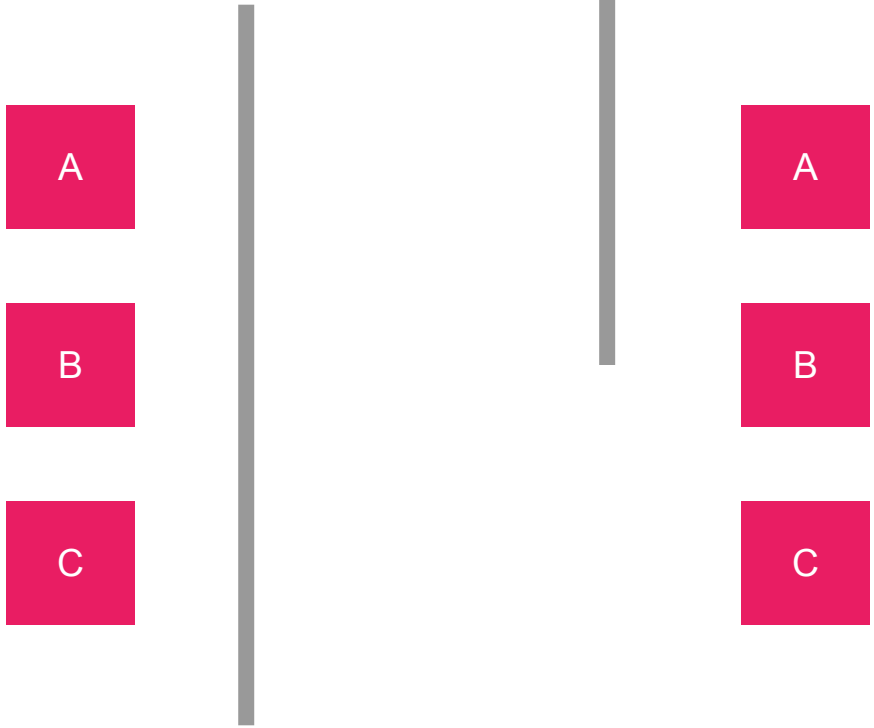
What is this solution lacking?

- A way to safely reset the barrier
  - Last thread to enter must allow others to exit (signal others)
  - Last thread to exit must prevent others from exiting (clean up)
- A way to prevent threads from entering while others are exiting
  - Last thread to enter must prevent others from entering
  - Last thread to exit must allow others to enter
  - 2 barriers?


A
D
D


A
B
C
D

Two Barriers



```
bool wait() {
  lock.lock();
  lock.unlock();
  return last;
}
```

```
bool wait() {
  lock.lock();
  while (threadsExited < numThreads)
    canEnterBarrier.wait(lock);

  lock.unlock();
  return last;
}
```

Wait for previous threads to clear out

```
bool wait() {
  lock.lock();
  while (threadsExited < numThreads)              Wait for previous
    canEnterBarrier.wait(lock);                   threads to clear out
  threadsEntered++;
  bool last = threads_entered == numThreads;









  lock.unlock();
  return last;
}
```

```
bool wait() {
  lock.lock();
  while (threadsExited < numThreads)
    canEnterBarrier.wait(lock);
  threadsEntered++;
  bool last = threads_entered == numThreads;
  while (threadsEntered < numThreads)
    canExitBarrier.wait(lock);









  lock.unlock();
  return last;
}
```

Wait for previous threads to clear out

Wait for all threads to show up

```
bool wait() {
  lock.lock();
  while (threadsExited < numThreads)
    canEnterBarrier.wait(lock);
  threadsEntered++;
  bool last = threads_entered == numThreads;
  while (threadsEntered < numThreads)
    canExitBarrier.wait(lock);
  if (last) {
    threadsExited = 0;
    canExitBarrier.broadcast();
  }



  lock.unlock();
  return last;
}
```

Wait for previous threads to clear out

Wait for all threads to show up

Last thread to enter signals others to leave

```
bool wait() {
  lock.lock();
  while (threadsExited < numThreads)
    canEnterBarrier.wait(lock);
  threadsEntered++;
  bool last = threads_entered == numThreads;
  while (threadsEntered < numThreads)
    canExitBarrier.wait(lock);
  if (last) {
    threadsExited = 0;
    canExitBarrier.broadcast();
  }
  threadsExited++;
  if (threadsExited == numThreads) {
     threadsEntered = 0;
     canEnterBarrier.broadcast();
  }
  lock.unlock();
  return last;
}
```

Wait for previous threads to clear out

Wait for all threads to show up

Last thread to enter signals others to leave

Last thread to leave cleans up for next use

Questions?